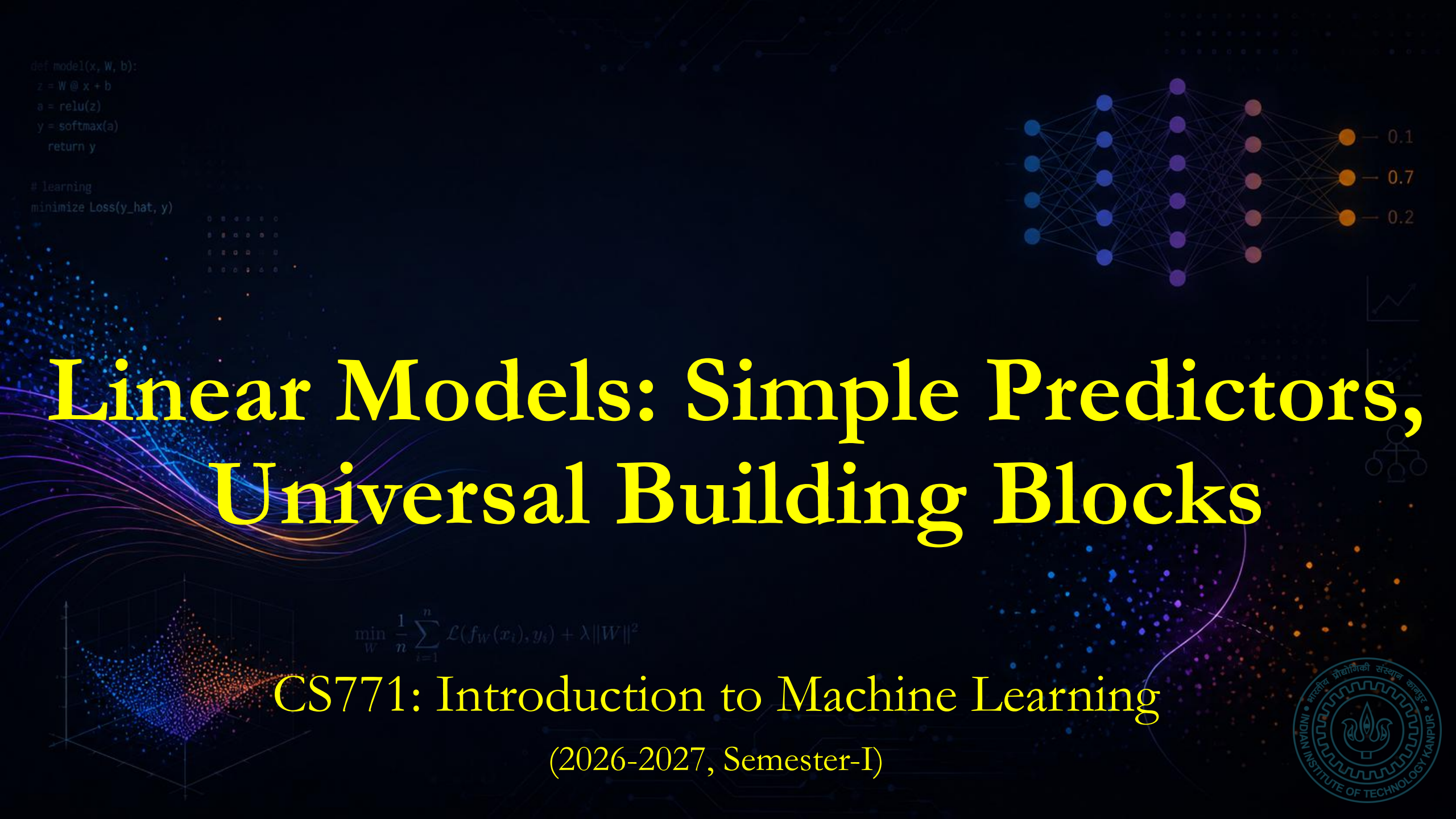


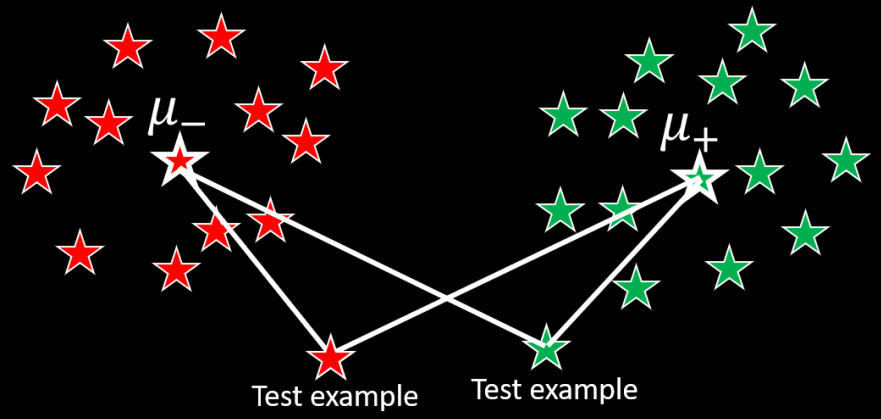
Linear Models: Simple Predictors, Universal Building Blocks

CS771: Introduction to Machine Learning

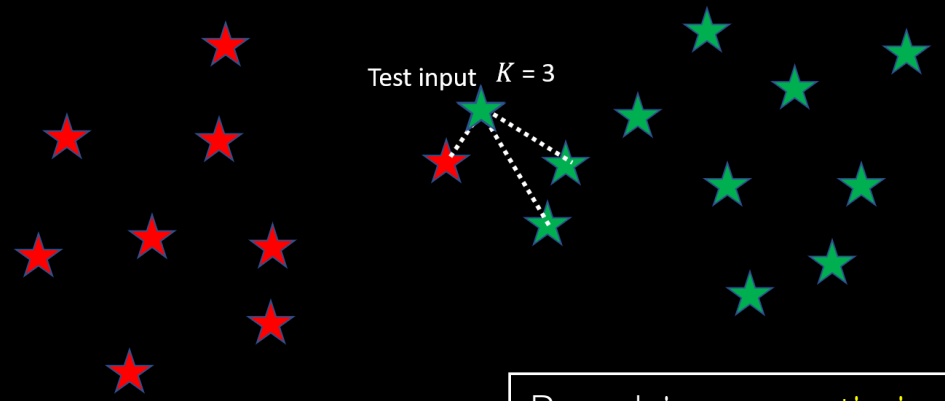
(2026-2027, Semester-I)

Story So Far, and What Lies Ahead..

Predicting by Comparing with Prototypes



Predicting by Finding Neighbors

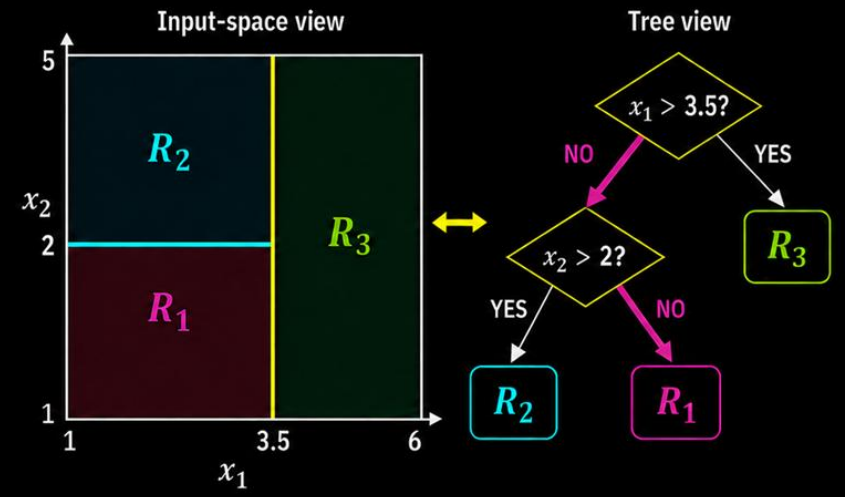


Up Next..

By solving an optimization problem

Predicting by Learning the underlying Function

Predicting by Divide and Conquer and Testing Simple Rules



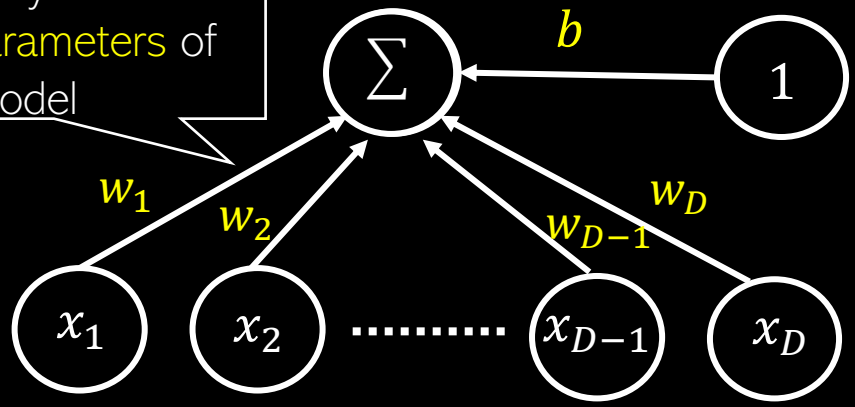


A Simple Function: Linear Model

$$y = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{j=1}^{D} w_j x_j + b$$

The weights and bias are collectively referred to as the parameters of the linear model

These are unknown and their optimal values are found by solving an optimization problem using the given training data



.. where $\boldsymbol{x} \in \mathbb{R}^D, y \in \mathbb{R}$ denote the input and output, respectively

and $\boldsymbol{w} \in \mathbb{R}^D, b \in \mathbb{R}$ denote the weights and bias, respectively

Writing $\tilde{\boldsymbol{x}} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_D \\ 1 \end{bmatrix} \in \mathbb{R}^{D+1}$ $\tilde{\boldsymbol{w}} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \\ b \end{bmatrix} \in \mathbb{R}^{D+1}$ ➡ $y = \tilde{\boldsymbol{w}}^\top \tilde{\boldsymbol{x}}$

Notational brevity by writing the linear model without explicitly showing the bias term

Will sometimes omit the tilde notation and often simply write $y = \boldsymbol{w}^\top \boldsymbol{x}$ assuming $\boldsymbol{x}$ and $\boldsymbol{w}$ are the augmented versions

Linear Model for Multiple Outputs

M different quantities to be predicted for the input $\boldsymbol{x}$

In some problems, the output is a vector, e.g., $\boldsymbol{y} \in \mathbb{R}^M$ or $\boldsymbol{y} \in \{0,1\}^M$

For such problems, we have $\boldsymbol{M}$ sets of weights and biases $\{\boldsymbol{w}_i, b_i\}_{i=1}^M$

$$y_1 = \boldsymbol{w}_1^\top \boldsymbol{x} + b_1$$
$$y_2 = \boldsymbol{w}_2^\top \boldsymbol{x} + b_2$$
$$\vdots$$
$$y_{M-1} = \boldsymbol{w}_{M-1}^\top \boldsymbol{x} + b_{M-1}$$
$$y_M = \boldsymbol{w}_M^\top \boldsymbol{x} + b_M$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{M-1} \\ y_M \end{bmatrix} = \begin{bmatrix} — & \boldsymbol{w}_1^\top & — \\ — & \boldsymbol{w}_2^\top & — \\ & \vdots & \\ — & \boldsymbol{w}_{M-1}^\top & — \\ — & \boldsymbol{w}_M^\top & — \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{D-1} \\ x_D \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_{M-1} \\ b_M \end{bmatrix}$$

Denotes a $D \times M$ weight matrix

$$\boldsymbol{y} = \boldsymbol{W}^\top \boldsymbol{x} + \boldsymbol{b}$$

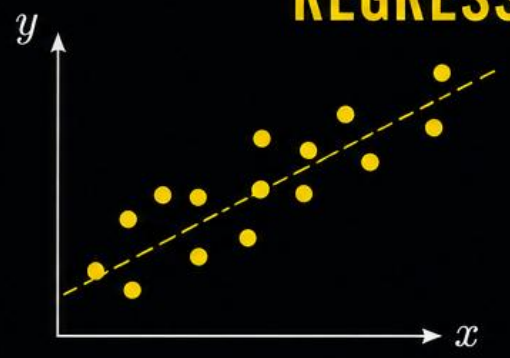
Linear models with vector outputs are very common in ML, such as multi-output regression, multi-class or multi-label classification, computing embeddings, etc

In deep neural networks, each hidden layer can also be seen as denoting a vector output



Linear Models Are Ubiquitous. Some Examples..

REGRESSION

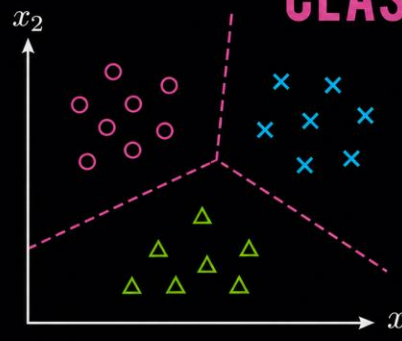


$$y = \mathbf{w}^\top \mathbf{x} + b$$

scalar prediction

$\mathbf{x} \in \mathbb{R}^D$ $\mathbf{w} \in \mathbb{R}^D$ $b \in \mathbb{R}$ $y \in \mathbb{R}$

CLASSIFICATION



$$z = W^\top \mathbf{x} + b$$

$$P(y = k \mid \mathbf{x}) = [\,\mathrm{softmax}(W^\top \mathbf{x} + b)\,]_k = \frac{\exp(z_k)}{\sum_{j=1}^{C} \exp(z_j)}, \quad k = 1, \ldots, C$$

$W \in \mathbb{R}^{D \times C}$ $b \in \mathbb{R}^C$

$P(y = k \mid \mathbf{x})$ = probability of class k

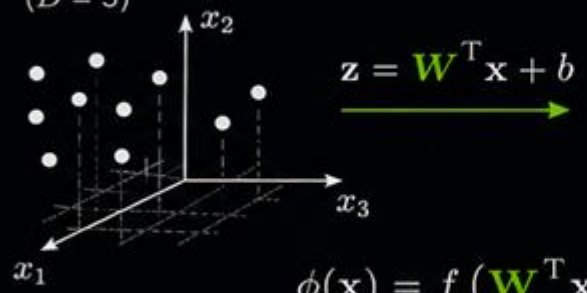
one score (probability) per class

$\mathbf{x} \in \mathbb{R}^D$ $W \in \mathbb{R}^{D \times C}$ $b \in \mathbb{R}^C$ $P(y \mid \mathbf{x}) \in \mathbb{R}^C$

REPRESENTATIONS (EMBEDDING)

input $x \in \mathbb{R}^D$

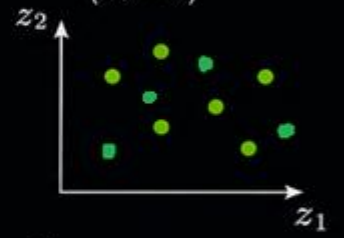
$(D = 3)$



$$\mathbf{z} = W^\top \mathbf{x} + b$$

embedding $\mathbf{z} = \phi(\mathbf{x}) \in \mathbb{R}^m$

$(m = 2)$

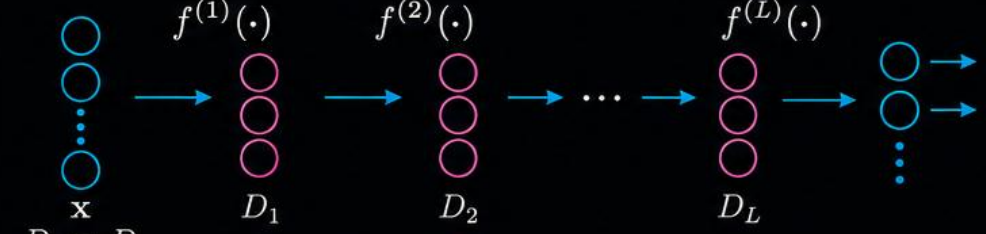


$$\phi(\mathbf{x}) = f\left(\mathbf{W}^\top \mathbf{x} + b\right)$$

learned embedding

$\mathbf{x} \in \mathbb{R}^D$ $\mathbf{W} \in \mathbb{R}^{D \times m}$ $b \in \mathbb{R}^m$ $z \in \mathbb{R}^m$

NEURAL NETWORKS (DEEP)



$$h^{(\ell)} = f\left(\mathbf{W}^{(\ell)\top} h^{(\ell-1)} + b^{(\ell)}\right)$$

$h^{(0)} = \mathbf{x}$ $\ell = 1$ $\ell = 2$ $\cdots$ $\ell = L$ $y = h^{(L)}$ D_L

$W^{(\ell)} \in \mathbb{R}^{D_{\ell-1} \times D_\ell}$ $b^{(\ell)} \in \mathbb{R}^{D_\ell}$

What is meant by "Linear" in a Linear Model?

Consider a linear model for an ML problem (e.g., regression) with D features

output is a linear combination of the D input features

$$y = \boldsymbol{w}^\top \boldsymbol{x} = \sum_{j=1}^{D} w_j x_j$$

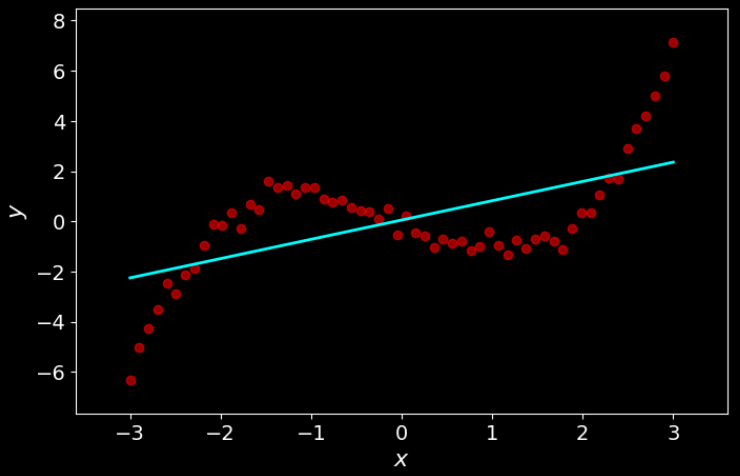
Omitted the bias term b for simplicity

$$y = f(x) = \boldsymbol{w}^\top \boldsymbol{x}$$

Linear means is linear in the inputs: output is a linear function of inputs



Does it mean linear models are useless when learning nonlinear functions like the nonlinear regression problem shown in this figure?



No - if the input features $\boldsymbol{x} \in \mathbb{R}^D$ are nonlinearly transformed to $\boldsymbol{\phi}(\boldsymbol{x}) \in \mathbb{R}^L$ and a linear model $y = \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x})$ is learned on the transformed inputs $\boldsymbol{\phi}(\boldsymbol{x})$

The new features $\boldsymbol{\phi}(\boldsymbol{x})$ can be hand-crafted or extracted using a learned feature extractor such as deep neural network



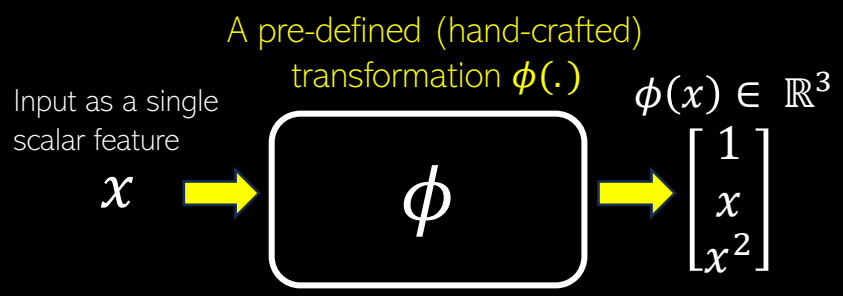


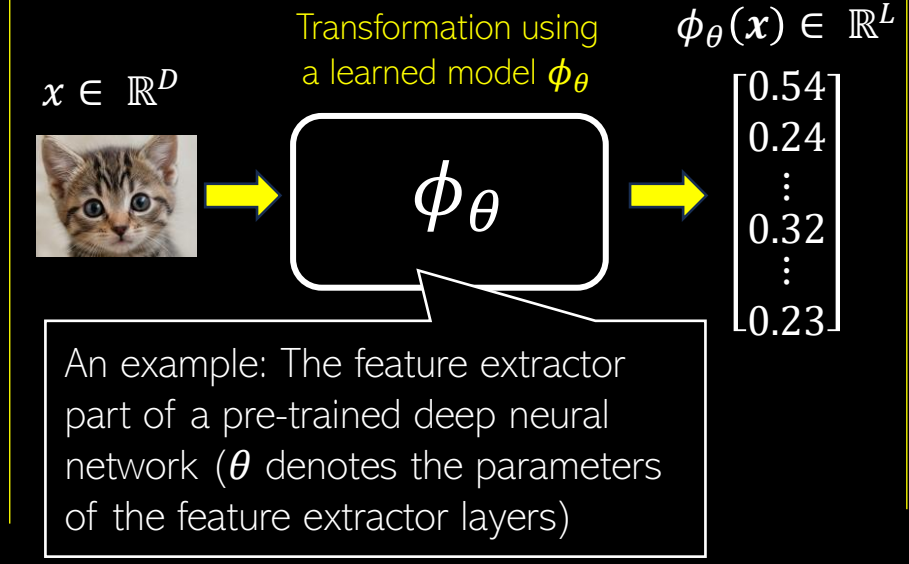
Linear Models on Nonlinear Features

Very powerful idea. A lot of nonlinear models (including deep learning) use it.

Idea: Transform input $\boldsymbol{x} \in \mathbb{R}^D$ to new feature representation $\boldsymbol{\phi(x)} \in \mathbb{R}^L$

The nonlinear features $\boldsymbol{\phi(x)}$ can be hand-crafted or learned





$$y = \boldsymbol{w}^\top \boldsymbol{\phi(x)}$$

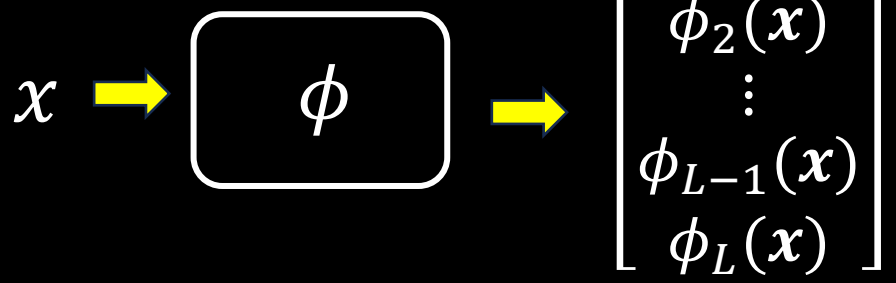
A linear model in the new features $\boldsymbol{\phi(x)}$

But nonlinear in the original features $\boldsymbol{x}$

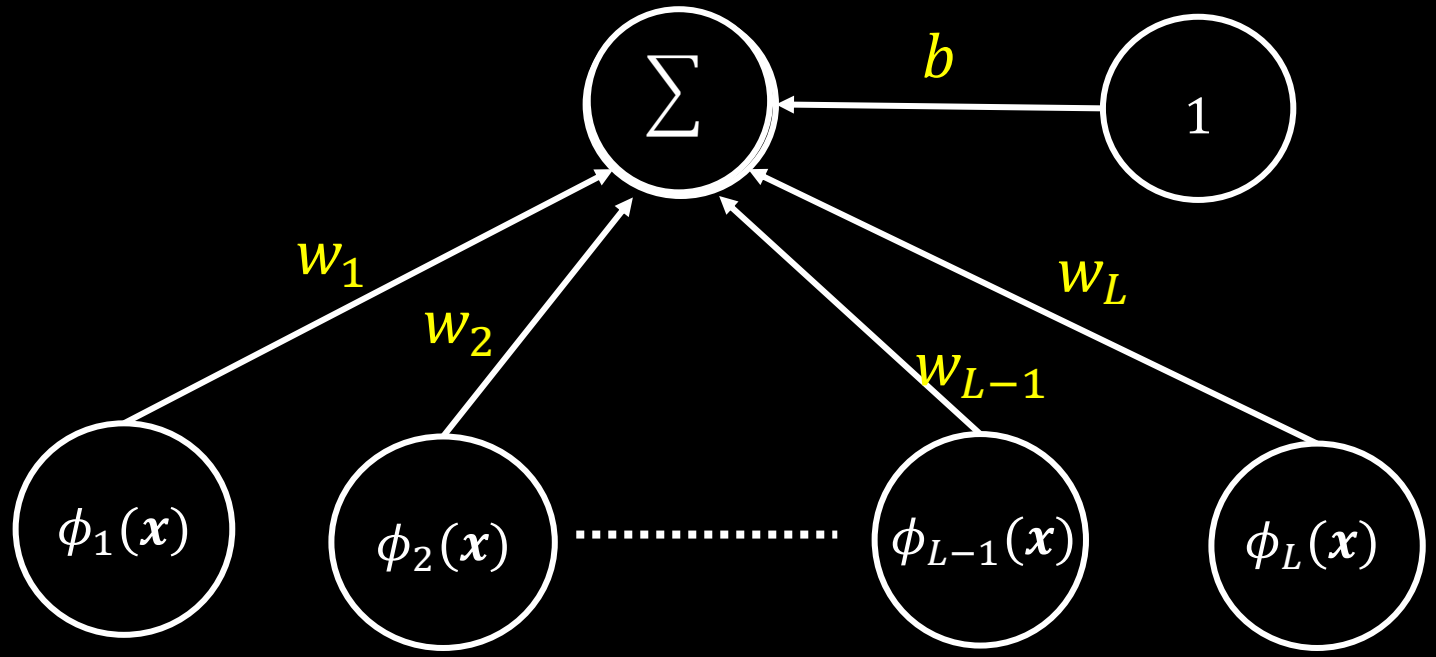
Linear Models on Nonlinear Features

Nonlinear Transformation of the original inputs $\boldsymbol{x}$

$$\phi(\boldsymbol{x}) \in \mathbb{R}^L$$



Learning a linear model on the transformed nonlinear features

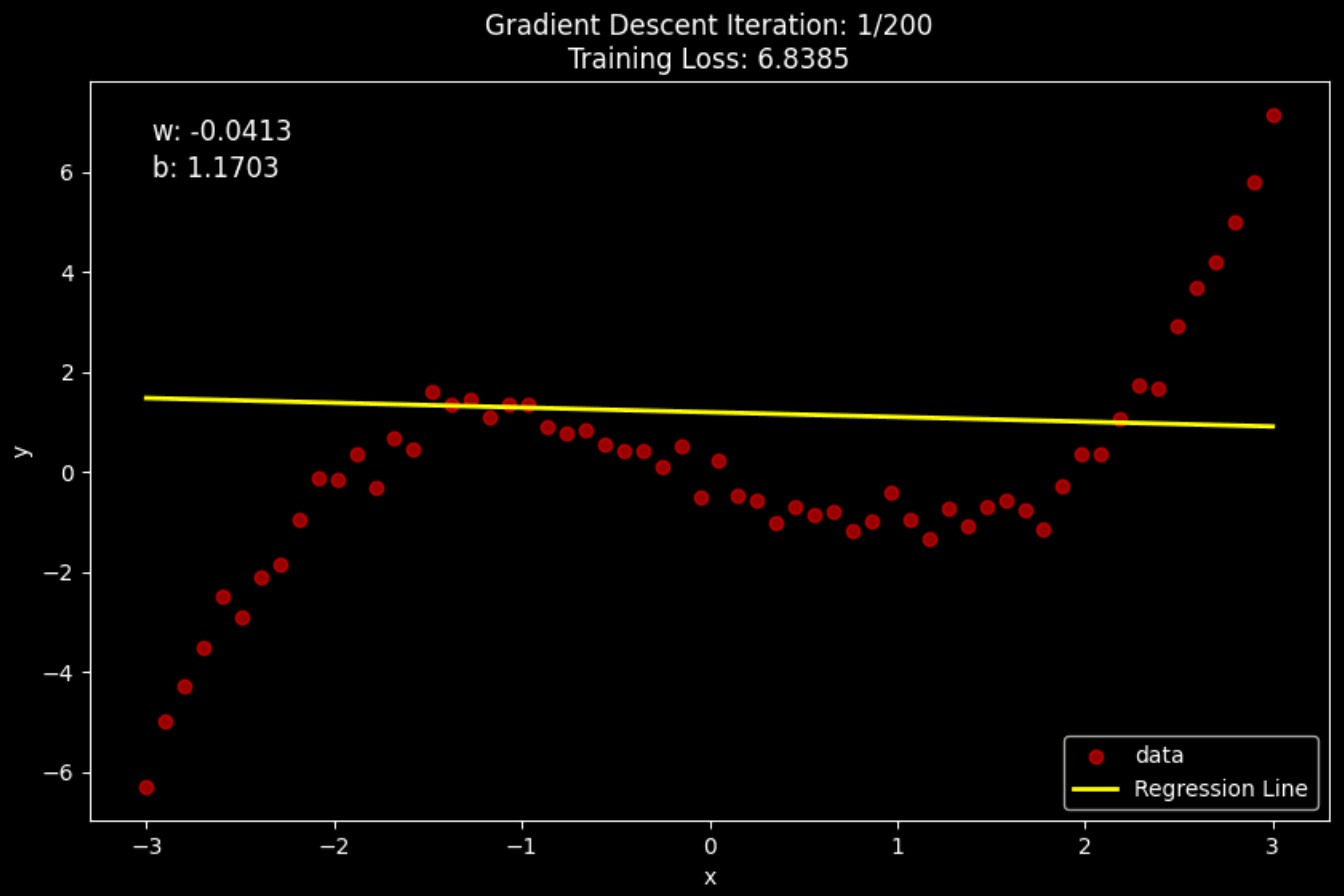


Linear Model in Action: Original Features

Training data approximately follows a degree-3 polynomial

Using original inputs (x) provides a poor fit

Model training using Gradient Descent (will study later)



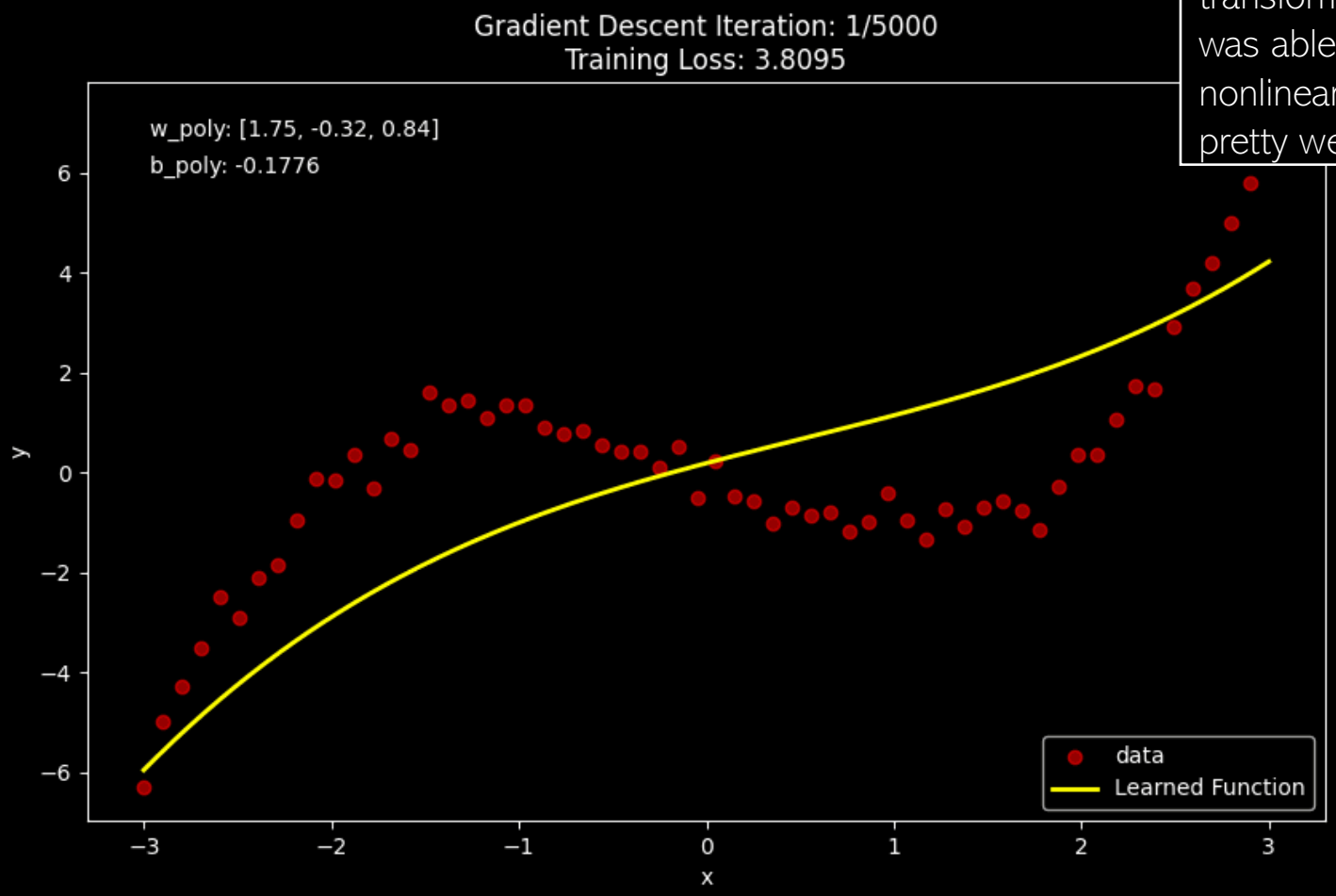
Linear Model in Action: Transformed Features

Training data approximately follows a degree-3 polynomial

Using the below transformation helps!

$$x \rightarrow \phi(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ x^3 \end{bmatrix}$$

Model training using Gradient Descent (will study later)



So even the linear model with transformed features was able to learn the nonlinear function pretty well





Learning The Parameters of A Linear Model

Consider the simplest linear model: linear regression with scalar outputs

$$y = \boldsymbol{w}^\top \boldsymbol{x} + b$$

Assume we have N training examples $\{(\boldsymbol{x}_n, y_n)\}_{n=1}^N$ to learn the model

Learning here means finding the "optimal" values of $\boldsymbol{w}$ and b

The optimal $\boldsymbol{w}$ and b should satisfy, for each training example $(\boldsymbol{x}_n, y_n)$

Ground truth output

$$y_n \approx \boldsymbol{w}^\top \boldsymbol{x}_n + b$$

Prediction $\hat{y}_n$ of the linear model with parameters $\boldsymbol{w}$ and b

Caution: The above alone is not sufficient. Reason: Possible overfitting



Learning The Parameters of A Linear Model

Ignoring overfitting issue, for now, we want the following to be small

"Residual" on prediction for training example n

$$r_n = y_n - (w^\top x_n + b) \qquad n = 1,2,\dots,N$$

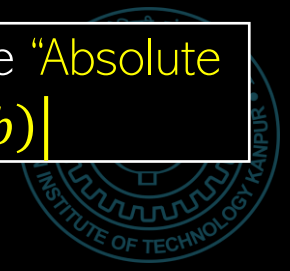
For any $\boldsymbol{w}$ and $\boldsymbol{b}$ we can define the total error or "loss" on training data as

$$\mathcal{L}(w,b) = \sum_{n=1}^{N} r_n^2 = \sum_{n=1}^{N} \left(y_n - (w^\top x_n + b)\right)^2$$

Loss functions other than squared loss or absolute loss can also be used (will discuss later)

The popular "Squared Loss"

Another option could have been the "Absolute Loss" defined as $\left|y_n - (w^\top x_n + b)\right|$



Learning The Parameters of A Linear Model

We estimate the optimal $\boldsymbol{w}$ and $\boldsymbol{b}$ by solving an optimization problem

Minimizing the loss function w.r.t. parameters $\boldsymbol{w}$ and $\boldsymbol{b}$

$$(w_{\text{opt}}, b_{\text{opt}}) = \arg\min_{w,b} \mathcal{L}(w, b)$$

(subject to some constraints on $\boldsymbol{w}, \boldsymbol{b}$)

Needed to ensure that there is no overfitting on training data

For now, let's ignore this. We will discuss the remedies later

If using squared loss, we will be solving the optimization problem

$$(w_{\text{opt}}, b_{\text{opt}}) = \arg\min_{w,b} \sum_{n=1}^{N} \left(y_n - (w^\top x_n + b)\right)^2$$



Solving Optimization Problems in ML

This is where calculus and optimization enters ML

We can try to solve them in closed form to find the optimal parameters

Solving in closed form is rarely possible except in some simple cases

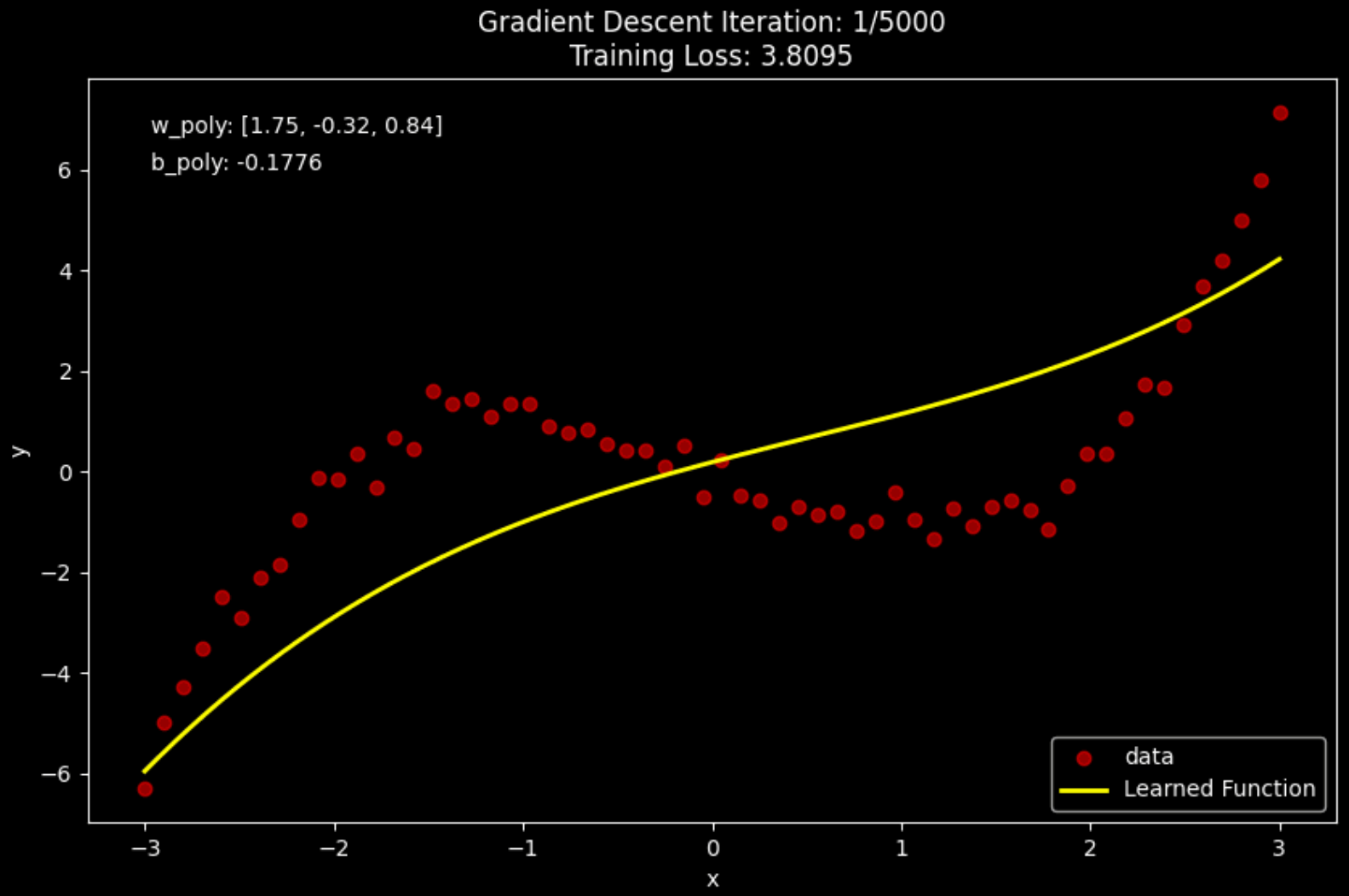
Iterative schemes for optimization are more common. In plain English:

Keep trying different values of the parameters iteratively (and strategically) till the loss function that we are minimizing becomes small enough

Basic idea behind iterative optimization algorithms such as Gradient Descent and its variants



Iterative Optimization: Illustrated on Toy Data



Next Class

Different methods for solving optimization problems in ML

Methods to prevent overfitting

